

The ML Mental Model — Supervised Learning, the 8-Step Workflow & Train-Test Split

Phase 2 starts today. By the end of this session you will understand the three types of machine learning, memorise the 8-step ML workflow that professionals use on every project, perform a proper train-test split, and build your very first working ML model using scikit-learn. Every model you build from Day 14 to Day 28 follows exactly this workflow.

Install scikit-learn before class starts:

Mac (venv activated): `pip install scikit-learn`

Windows: `pip install scikit-learn`

Linux: `pip3 install scikit-learn`

Verify: `python3 -c "import sklearn; print(sklearn.__version__)"`

You should see something like: 1.4.2

SECTION 01 The Three Types of Machine Learning

Before you can choose a model, you need to know which type of ML problem you are solving. There are three types. Confusing them leads to using the wrong tools completely.

Type 1 — Supervised Learning

Supervised Learning

What it is: You give the algorithm labelled examples — inputs paired with the correct output. The algorithm learns the mapping from input to output. At prediction time, it applies that learned mapping to new, unseen inputs.

Real-life analogy: Think of a student learning from a textbook with an answer key. Every practice problem shows the question (input) AND the correct answer (output). The student studies these pairs and learns the pattern. In an exam, they apply that pattern to new questions. The teacher supervises the learning with correct answers.

In Python: `model.fit(X_train, y_train)` — X is inputs, y is correct outputs (labels)

Supervised learning has two sub-types:

Sub-type	Target variable	Examples
Regression	A continuous number	Predict house price (Rs. 45 lakhs), temperature (36.5°), stock price
Classification	A category / class	Spam or not spam, disease or healthy, which digit 0-9

- Regression examples: Predict the salary of a new employee. Predict tomorrow's rainfall in mm. Predict a student's exam score from study hours.
- Classification examples: Classify an email as spam or not spam. Detect whether a transaction is fraudulent. Diagnose whether a tumour is malignant or benign.

Type 2 — Unsupervised Learning

Unsupervised Learning

What it is: You give the algorithm data WITHOUT any labels or correct answers. The algorithm finds hidden patterns, groups, or structure in the data entirely on its own. There is no teacher. The algorithm is exploring.

Real-life analogy: Think of a new librarian given 10,000 books with no categories written on them. They start reading and naturally group similar books together: fiction, history, science, cooking. Nobody told them the categories. They discovered the structure themselves. That is unsupervised learning.

In Python: `KMeans().fit(X)` — no `y` needed. The model finds groups on its own.

- Clustering: Group customers by purchasing behaviour. Group news articles by topic. Segment patients by symptoms.
- Dimensionality reduction: Compress a 500-feature dataset to 10 features while keeping most information. Used before training other models.
- Anomaly detection: Find the unusual data points that do not fit any group. Used in fraud detection, quality control.

Type 3 — Reinforcement Learning

Reinforcement Learning

What it is: An agent learns by interacting with an environment. It takes actions, receives rewards or penalties, and gradually learns which actions lead to the best cumulative reward. There is no dataset of labelled examples — the agent generates its own experience.

Real-life analogy: Training a dog. You do not give it a manual. You give it a treat when it sits on command (reward) and no treat when it does not (neutral). Over many repetitions, the dog learns which behaviours earn treats. The dog is the agent. The treat is the reward. The environment is the room.

In Python: Not covered in depth this program — requires separate foundations in RL theory.

- Game playing: AlphaGo (beats world Go champion), OpenAI Five (beats professional Dota 2 players)
- Robotics: Teaching a robotic arm to grasp objects by trial and error
- Recommendation systems: Learning which content keeps users engaged longer

Focus for this program: Supervised Learning (Days 14-28) and Unsupervised (Days 24-25). Reinforcement Learning is a separate field requiring its own deep study. The most in-demand industry jobs use supervised learning 90% of the time.

SECTION 02 The 8-Step ML Workflow — Memorise This

Every professional ML project, from a student assignment to a Google production system, follows this sequence. Not always in a perfectly linear way, but always visits every step. Learn this so well that you can draw it from memory.

1

Define the problem clearly

What exactly are you predicting? What is success? Who will use this model and how? A vague problem gives a useless model. Write the problem as a one-sentence statement: 'Predict whether a customer will churn in the next 30 days based on their usage history.'

2

Collect and understand the data

Get the raw data. Run `df.info()`, `df.describe()`, `df.isnull().sum()`. Understand what each column means, what the target variable is, and what patterns might exist. This is the EDA you practised in Phase 1.

3

Clean and preprocess the data

Handle missing values. Remove or fix outliers. Fix data types. Standardise text. This is everything from Day 8. A model trained on dirty data gives dirty predictions.

4

Engineer features

Create new columns that might help the model. Turn a date into day-of-week. Turn a text column into numeric categories. Combine two columns. This is the single highest-leverage step in improving model performance. Covered in depth on Day 21.

5

Split the data into training and test sets

Set aside a portion of data (typically 20%) that the model will NEVER see during training. This held-out set is used to measure how well the model generalises to new data. We cover this in depth in Section 03 today.

6

Choose and train a model

Select an algorithm appropriate for the problem (regression vs classification, linear vs tree-based). Call `model.fit(X_train, y_train)`. The algorithm runs gradient descent or its equivalent and learns the parameters.

7 Evaluate the model

Measure performance on the TEST set (data the model never saw). Use the right metrics: RMSE/ R^2 for regression, accuracy/precision/recall/F1/AUC for classification. Compare to a simple baseline. Is this model actually useful?

8 Tune, iterate, and deploy

Adjust hyperparameters. Try different algorithms. Do more feature engineering. Go back to earlier steps if needed. Once satisfied, deploy the model as an API or embed it in an application so real users can get predictions.

The most important insight about this workflow:

It is a LOOP, not a straight line.

After Step 7 (evaluation), you often go back to Step 3 (more cleaning), Step 4 (better features), or Step 6 (different model).

Professional data scientists iterate through this loop many times before finding a model good enough to deploy.

A junior mistake: run the model once, get 72% accuracy, call it done.

A senior move: ask 'why 72%?', go back, improve features, reach 88%.

SECTION 03 Train-Test Split — The Honest Way to Measure a Model

The train-test split is the most fundamental concept in ML evaluation. Without it, you have no way of knowing if your model actually learned something useful or just memorised the training data.

The problem it solves — why you cannot test on training data

Imagine a student who memorises the exact textbook questions and their answers, word for word. On an exam using those same questions, they score 100%. Does this mean they understood the subject? No. They just memorised. Give them new questions and they fail completely.

A model tested on its own training data gives a fake, inflated accuracy score.

Example: A model trained on 1000 customer records and tested on those SAME 1000 records might show 99% accuracy — because it essentially memorised them.

When deployed to serve predictions on NEW customers, it might only get 61%.
That 38-point gap is the gap between training accuracy and real-world accuracy.

The train-test split forces you to measure performance on data the model has NEVER seen. This gives you an honest estimate of real-world performance.

How train-test split works

Your full dataset: 1000 rows

```
|----- Training set (800 rows, 80%) -----|-- Test set (200  
rows, 20%) --|
```

Model learns from these 800 rows
these during training

```
model.fit(X_train, y_train)  
-> compare to y_test
```

Model NEVER sees

```
model.predict(X_test)
```

High accuracy here = model learned SOMETHING
model GENERALISES

High accuracy here =

Low accuracy here = underfitting (too simple)
poor real-world performance

Low accuracy here =

Very high accuracy but low test accuracy = overfitting (memorised
training data)

```
from sklearn.model_selection import train_test_split  
import pandas as pd  
import numpy as np  
  
# Load a dataset  
from sklearn.datasets import load_iris  
iris = load_iris()  
X = iris.data      # Features: sepal length, sepal width, petal length,  
petal width  
y = iris.target    # Labels: 0=Setosa, 1=Versicolor, 2=Virginica  
  
print(f"Full dataset: X={X.shape}, y={y.shape}")  
  
# Perform train-test split  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size = 0.2,      # 20% goes to test set  
    random_state = 42,    # Seed for reproducibility  
    stratify = y          # Keep class proportions equal in both sets  
)  
  
print(f"Training set : X_train={X_train.shape}, y_train={y_train.shape}")
```

```

print(f"Test set      : X_test={X_test.shape}, y_test={y_test.shape}")

# Verify class distribution is balanced
print(f"\nClass distribution in full dataset:")
print(f"  Class 0: {np.sum(y==0)}, Class 1: {np.sum(y==1)}, Class 2: {np.sum(y==2)}")
print(f"Class distribution in training set:")
print(f"  Class 0: {np.sum(y_train==0)}, Class 1: {np.sum(y_train==1)}, Class 2: {np.sum(y_train==2)}")
print(f"Class distribution in test set:")
print(f"  Class 0: {np.sum(y_test==0)}, Class 1: {np.sum(y_test==1)}, Class 2: {np.sum(y_test==2)}")

```

Output:

```

Full dataset: X=(150, 4), y=(150,)
Training set : X_train=(120, 4), y_train=(120,)
Test set     : X_test=(30, 4), y_test=(30,)

```

```

Class distribution in full dataset:
  Class 0: 50, Class 1: 50, Class 2: 50
Class distribution in training set:
  Class 0: 40, Class 1: 40, Class 2: 40
Class distribution in test set:
  Class 0: 10, Class 1: 10, Class 2: 10

```

Three critical parameters of train_test_split:

`test_size = 0.2` — 20% held out for testing. Standard for most datasets.

Use 0.1 for very small datasets (<500 rows).

Use 0.3 for very large datasets (>100k rows).

`random_state = 42` — Fixes the random shuffle. Without this, every run gives a different split and different accuracy scores.

Use 42 by convention (any number works).

`stratify = y` — Ensures each class appears proportionally in both sets.

Essential for classification. Without this, your test set might accidentally have all examples of one class.

SECTION 04 Introduction to Scikit-Learn — The ML Toolkit

Scikit-learn (sklearn) is the most used machine learning library in Python. It provides a consistent interface for dozens of algorithms, preprocessing tools, and evaluation metrics. Learning the sklearn pattern once means you can use any algorithm in the library.

The universal sklearn pattern — the same four steps for EVERY algorithm:

Step 1: Import the class

```
from sklearn.linear_model import LogisticRegression
```

Step 2: Instantiate with hyperparameters

```
model = LogisticRegression(max_iter=1000)
```

Step 3: Train on training data

```
model.fit(X_train, y_train)
```

Step 4: Predict and evaluate on test data

```
y_pred = model.predict(X_test)
```

```
score = model.score(X_test, y_test)
```

This exact same pattern works for Linear Regression, Decision Trees, Random Forest, XGBoost, SVM, KNN, and every other sklearn algorithm.

The sklearn ecosystem — what is available

Module	What it contains
sklearn.model_selection	train_test_split, cross_val_score, GridSearchCV
sklearn.linear_model	LinearRegression, LogisticRegression, Ridge, Lasso
sklearn.tree	DecisionTreeClassifier, DecisionTreeRegressor
sklearn.ensemble	RandomForestClassifier, GradientBoostingClassifier
sklearn.svm	SVC, SVR (Support Vector Machine)
sklearn.neighbors	KNeighborsClassifier, KNeighborsRegressor
sklearn.naive_bayes	GaussianNB, MultinomialNB
sklearn.preprocessing	StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder
sklearn.pipeline	Pipeline, ColumnTransformer
sklearn.metrics	accuracy_score, mean_squared_error, confusion_matrix, roc_auc_score
sklearn.datasets	load_iris, load_boston, make_classification, make_regression

Before building a real model, every professional establishes a baseline. The baseline is the simplest possible prediction strategy. Your real model must beat the baseline to be considered useful. This is a step most beginners skip, and it costs them later.

Why you always need a baseline:

Imagine a dataset where 95% of transactions are NOT fraud.

A model that predicts 'not fraud' for EVERYTHING gets 95% accuracy.

Does that mean it is a good model? Absolutely not. It never detects any fraud.

Without a baseline, 95% sounds impressive.

With a baseline, you know 95% is the floor, not the ceiling.

Your model must do better than this trivial strategy to be worth anything.

```
from sklearn.dummy import DummyClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
import numpy as np

# Load Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Strategy 1: Always predict the most common class
dummy_most_freq = DummyClassifier(strategy='most_frequent')
dummy_most_freq.fit(X_train, y_train)
y_pred_mf = dummy_most_freq.predict(X_test)
acc_mf = accuracy_score(y_test, y_pred_mf)

# Strategy 2: Predict randomly based on class proportions
dummy_random = DummyClassifier(strategy='stratified', random_state=42)
dummy_random.fit(X_train, y_train)
y_pred_r = dummy_random.predict(X_test)
acc_r = accuracy_score(y_test, y_pred_r)

print(f"Baseline (most frequent class) accuracy: {acc_mf:.4f} ({acc_mf*100:.1f}%)")
print(f"Baseline (random stratified) accuracy : {acc_r:.4f} ({acc_r*100:.1f}%)")
print()
print("Any real model must beat these numbers to be considered useful.")
print(f"Most frequent class: {iris.target_names[np.bincount(y).argmax()]})")
```

Output:

Baseline (most frequent class) accuracy: 0.3333 (33.3%)

Baseline (random stratified) accuracy : 0.3333 (33.3%)

Any real model must beat these numbers to be considered useful.

Most frequent class: setosa

The Iris dataset has 3 equal classes. Guessing randomly gives 33.3%. Any real model must do better than 33.3% to be worth using. If your model gets 60%, you can say it is almost twice as good as random guessing. If it gets 97%, you have done excellent work.

SECTION 06 The Complete Workflow in Code — End to End

Now we put Steps 1-8 together in one clean script. We will use the Iris dataset and three different datasets to see the pattern in action. Tomorrow we replace DummyClassifier with real algorithms. Today we understand the plumbing.

```
# =====
# Complete 8-Step ML Workflow Demo
# =====

import numpy as np
import pandas as pd
from sklearn.datasets import load_iris, load_wine
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, classification_report

def run_ml_workflow(dataset_name, X, y, feature_names, target_names):
    """
    Runs the full 8-step ML workflow on any dataset.
    This exact function structure will be reused throughout Phase 2.
    """
    print(f"\n{'='*60}")
    print(f"DATASET: {dataset_name}")
    print(f"{'='*60}")

    # — STEP 1: Define the problem —————
    print(f"\nStep 1 – Problem:")
    print(f"  Classify samples into {len(target_names)} classes: {list(target_names)}")
    print(f"  Using {len(feature_names)} features: {list(feature_names)}")

    # — STEP 2: Understand the data —————
    print(f"\nStep 2 – Data overview:")
    print(f"  Total samples : {X.shape[0]}")
    print(f"  Total features : {X.shape[1]}")
```

```

class_counts = np.bincount(y)
for i, (name, count) in enumerate(zip(target_names, class_counts)):
    print(f" Class {i} ({name}): {count} samples
({count/len(y)*100:.0f}%)")

# — STEP 3-4: Assuming data is already clean for built-in datasets
print(f"\nStep 3-4 – Data is clean (built-in sklearn dataset)")

# — STEP 5: Train-test split —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"\nStep 5 – Train-test split:")
print(f" Training samples: {X_train.shape[0]}")
print(f" Test samples      : {X_test.shape[0]}")

# — STEP 6: Train model (Dummy baseline for now) —————
model = DummyClassifier(strategy='most_frequent', random_state=42)
model.fit(X_train, y_train)
print(f"\nStep 6 – Model trained: DummyClassifier (baseline)")

# — STEP 7: Evaluate —————
y_pred = model.predict(X_test)
train_acc = accuracy_score(y_train, model.predict(X_train))
test_acc = accuracy_score(y_test, y_pred)

print(f"\nStep 7 – Evaluation:")
print(f" Training accuracy : {train_acc:.4f} ({train_acc*100:.1f}%)")
print(f" Test accuracy      : {test_acc:.4f} ({test_acc*100:.1f}%)")
print(f"\n This baseline must be BEATEN by any real model.")

return X_train, X_test, y_train, y_test, test_acc

# — Run on Iris —————
iris = load_iris()
run_ml_workflow('Iris Flowers', iris.data, iris.target,
                iris.feature_names, iris.target_names)

# — Run on Wine —————
wine = load_wine()
run_ml_workflow('Wine Classification', wine.data, wine.target,
                wine.feature_names, wine.target_names)

```

Output:

```

=====
DATASET: Iris Flowers
=====

```

Step 1 – Problem:

```

Classify samples into 3 classes: ['setosa', 'versicolor', 'virginica']
Using 4 features: ['sepal length (cm)', 'sepal width (cm)', ...]

```

```

Step 2 – Data overview:
  Total samples   : 150
  Total features  : 4
  Class 0 (setosa)    : 50 samples (33%)
  Class 1 (versicolor): 50 samples (33%)
  Class 2 (virginica) : 50 samples (33%)

Step 3-4 – Data is clean (built-in sklearn dataset)

Step 5 – Train-test split:
  Training samples: 120
  Test samples    : 30

Step 6 – Model trained: DummyClassifier (baseline)

Step 7 – Evaluation:
  Training accuracy : 0.3333 (33.3%)
  Test accuracy      : 0.3333 (33.3%)

  This baseline must be BEATEN by any real model.

```

SECTION 07 Understanding X and y — The ML Naming Convention

Throughout Phase 2 you will see X and y constantly. Understanding exactly what they are and why they are named this way prevents a huge amount of confusion.

Variable	What it is	Shape
X (capital)	The feature matrix. All input columns. What the model uses to make predictions.	(n_samples, n_features)
y (lowercase)	The target vector. The column we are trying to predict. The answer.	(n_samples,)
X_train	The feature matrix for training (80% of data).	(n_train, n_features)
X_test	The feature matrix for testing (20% of data).	(n_test, n_features)
y_train	The target values for training rows.	(n_train,)
y_test	The true target values for test rows. Used ONLY for evaluation.	(n_test,)
y_pred	The predictions the model made for X_test.	(n_test,)

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

```

```

# Example: Customer churn prediction
data = {
    'age' : [25, 35, 45, 28, 52, 33, 41, 29, 38, 47],
    'monthly_spend' : [500,1200,800,300,950,700,1100,450,600,850],
    'tenure_months' : [6, 24, 36, 3, 48, 18, 30, 9, 12, 42],
    'support_calls' : [1, 0, 2, 4, 0, 1, 0, 3, 2, 1],
    'churned' : [0, 0, 1, 1, 0, 0, 0, 1, 1, 0] # TARGET
}
df = pd.DataFrame(data)

# Separate features (X) from target (y)
X = df.drop(columns=['churned']) # Everything except the target
y = df['churned'] # Just the target column

print(f"X shape: {X.shape} (10 customers, 4 features each)")
print(f"y shape: {y.shape} (10 labels: 0=stayed, 1=churned)")
print()
print("X (features):")
print(X)
print()
print("y (target):")
print(y.values)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
print(f"\nX_train: {X_train.shape}, X_test: {X_test.shape}")
print(f"y_train: {y_train.shape}, y_test: {y_test.shape}")

```

Output:

```

X shape: (10, 4) (10 customers, 4 features each)
y shape: (10,) (10 labels: 0=stayed, 1=churned)

X (features):
   age  monthly_spend  tenure_months  support_calls
0   25             500              6              1
1   35            1200             24              0
...

y (target):
[0 0 1 1 0 0 0 1 1 0]

X_train: (7, 4), X_test: (3, 4)
y_train: (7,), y_test: (3,)

```

Accuracy is not always the right metric. You need to choose the metric that matches your business problem. A model with 99% accuracy can still be completely useless. Understanding this is what separates a thoughtful data scientist from a code-runner.

For Classification Problems

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)
import numpy as np

# Simulate predictions for a fraud detection model
# 1 = fraud, 0 = not fraud
y_true = np.array([0,0,0,0,1,0,0,1,0,0,0,1,0,0,0,0,1,0,0,0])
y_pred = np.array([0,0,0,0,1,0,0,0,0,0,0,0,1,0,1,0,0,1,0,0,0])
# Note: predicted 0 for one real fraud (missed!), 1 for one non-fraud
# (false alarm)

print("Confusion Matrix:")
cm = confusion_matrix(y_true, y_pred)
print(cm)
print("  [TN  FP]")
print("  [FN  TP]")

tn, fp, fn, tp = cm.ravel()
print(f"\n True Negatives   (TN): {tn}  - correctly said NOT fraud")
print(f" False Positives   (FP): {fp}  - wrongly said fraud (false alarm)")
print(f" False Negatives   (FN): {fn}  - missed actual fraud (dangerous!)")
print(f" True Positives    (TP): {tp}  - correctly caught fraud")

print(f"\nAccuracy   : {accuracy_score(y_true, y_pred):.4f}   (correct / total)")
print(f"Precision  : {precision_score(y_true, y_pred):.4f}   (of predicted fraud, how many were real?)")
print(f"Recall     : {recall_score(y_true, y_pred):.4f}   (of actual fraud, how many did we catch?)")
print(f"F1 Score   : {f1_score(y_true, y_pred):.4f}   (harmonic mean of precision and recall)")

print("\nFull Report:")
print(classification_report(y_true, y_pred, target_names=['Not Fraud', 'Fraud']))
```

Output:

Confusion Matrix:

```
[[14  1]
 [ 1  4]]
  [TN  FP]
  [FN  TP]
```

True Negatives (TN): 14 - correctly said NOT fraud

False Positives (FP): 1 - wrongly said fraud (false alarm)

False Negatives (FN): 1 — missed actual fraud (dangerous!)
True Positives (TP): 4 — correctly caught fraud

Accuracy : 0.9000 (correct / total)
Precision : 0.8000 (of predicted fraud, how many were real?)
Recall : 0.8000 (of actual fraud, how many did we catch?)
F1 Score : 0.8000 (harmonic mean of precision and recall)

Metric	Formula	When it matters most
Accuracy	$(TP+TN) / \text{Total}$	Balanced classes. Quick overall picture.
Precision	$TP / (TP+FP)$	When false alarms are costly. Spam filter: do not block real emails.
Recall	$TP / (TP+FN)$	When missing positives is costly. Disease detection: do not miss sick patients.
F1 Score	$2 * (P * R) / (P + R)$	Imbalanced classes. Balances precision and recall.
ROC-AUC	Area under ROC curve	Overall model quality. Higher = better at separating classes.

The precision vs recall trade-off — you must understand this for interviews:

Fraud detection: Missing a fraud (low recall) costs money. Prefer HIGH RECALL.

A false alarm just means a call to verify. That is acceptable.

Spam filter: A false alarm (low precision) blocks a real email. Prefer HIGH PRECISION.

Missing a spam email is annoying but not catastrophic.

Cancer screening: Missing a patient with cancer (low recall) is life-threatening.

Prefer HIGH RECALL. False alarms just lead to more tests.

The business context ALWAYS determines which metric to optimise for.

For Regression Problems

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

# Actual vs predicted house prices (Rs. lakhs)
y_true = np.array([45, 60, 35, 80, 55, 70, 40, 90, 50, 65])
y_pred = np.array([47, 58, 38, 76, 57, 72, 42, 88, 53, 63])

mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_true, y_pred)
```

```

r2      = r2_score(y_true, y_pred)

print(f"MSE   : {mse:.2f}")
print(f"RMSE  : {rmse:.2f} lakhs  <- off by Rs. {rmse:.2f} lakhs on
average")
print(f"MAE   : {mae:.2f} lakhs  <- simpler average error")
print(f"R2   : {r2:.4f}  <- model explains {r2*100:.1f}% of price
variation")

```

Output:

```

MSE   : 5.40
RMSE  : 2.32 lakhs  <- off by Rs. 2.32 lakhs on average
MAE   : 2.10 lakhs  <- simpler average error
R2   : 0.9738  <- model explains 97.4% of price variation

```

SECTION 09 Replicating the Workflow on Two More Datasets

Practice seeing the same workflow applied to different datasets. The pattern does not change. Only the data changes. This is the core skill.

```

# Applying the workflow to a regression problem
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Create a synthetic regression dataset
# 500 samples, 5 features, predicting a continuous number
X, y = make_regression(
    n_samples = 500,
    n_features = 5,
    noise      = 20,
    random_state = 42
)

print(f"Dataset: {X.shape[0]} samples, {X.shape[1]} features")
print(f"Target range: {y.min():.0f} to {y.max():.0f}")

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Baseline: always predict the mean of training targets
baseline = DummyRegressor(strategy='mean')
baseline.fit(X_train, y_train)

```

```

y_pred_base = baseline.predict(X_test)

# Evaluate baseline
rmse_base = np.sqrt(mean_squared_error(y_test, y_pred_base))
r2_base    = r2_score(y_test, y_pred_base)

print(f"\nBaseline (always predict mean):")
print(f"  RMSE : {rmse_base:.2f}")
print(f"  R2  : {r2_base:.4f}")
print(f"\nA real regression model must beat RMSE {rmse_base:.0f} and R2 {r2_base:.2f}")
print(f"Tomorrow we will: LinearRegression will crush this baseline.")

```

Output:

Dataset: 500 samples, 5 features
Target range: -474 to 487

Baseline (always predict mean):
 RMSE : 163.42
 R² : -0.0030

A real regression model must beat RMSE 163 and R² -0.00
Tomorrow we will: LinearRegression will crush this baseline.

SECTION 10 Interview Prep — Questions to Answer After Today

These questions come up in almost every ML interview, even for senior roles. They test whether you understand the fundamentals, not just the code.

Q — What is the difference between supervised, unsupervised, and reinforcement learning?

Supervised learning uses labelled data: each input has a correct output, and the model learns the mapping from inputs to outputs. Examples include spam detection and house price prediction. Unsupervised learning has no labels: the algorithm finds hidden structure, patterns, or groupings in the data on its own. Examples include customer segmentation and anomaly detection. Reinforcement learning has an agent that learns by trial and error through interactions with an environment, receiving rewards or penalties. Examples include game-playing AI and robot control. In industry, supervised learning dominates commercial applications, unsupervised is used for exploration and preprocessing, and reinforcement learning is mainly research and gaming.

Q — Why do we split data into training and test sets?

The training set is what the model learns from. The test set is held out and never used during training. We evaluate the model on the test set to get an honest estimate of how well it will perform on new, unseen data in production. If we evaluated on the training data, a model that simply memorised the training examples would score perfectly while failing completely on new

data. This overfitting problem would be invisible without a separate test set. The test set simulates what happens when the model is deployed and receives data it has never seen before.

Q — What is a baseline model and why do you always build one first?

A baseline model is the simplest possible prediction strategy that requires no learning. For classification, it might always predict the most common class. For regression, it always predicts the mean. A baseline gives you the floor: the minimum performance any model must exceed to be considered useful. Without a baseline, a number like 85% accuracy sounds good but might actually be terrible if guessing the majority class gives 84%. With a baseline, you can quantify exactly how much value your real model adds. Every professional ML project starts by establishing a baseline before spending time on complex models.

Q — When would you use precision vs recall as your main metric?

Precision and recall measure different types of errors. Precision is the proportion of positive predictions that are actually correct. Recall is the proportion of actual positives that the model correctly identified. For a spam filter, I prioritise precision: I do not want legitimate emails blocked, so I want every email flagged as spam to truly be spam, even if I miss some spam. For a cancer screening test, I prioritise recall: missing a real cancer case could be fatal, so I want to catch every positive case even if it means some false alarms that lead to further testing. The business consequence of false positives versus false negatives determines which metric to optimise.

SECTION 11 Words to Know

Term	Definition
Supervised learning	ML with labelled data. Model learns input → output mapping.
Unsupervised learning	ML without labels. Model finds hidden structure in data.
Reinforcement learning	Agent learns by interacting with environment and receiving rewards.
Regression	Supervised learning where target is a continuous number.
Classification	Supervised learning where target is a category or class.
X (feature matrix)	2D array of input features. Shape (n_samples, n_features).
y (target vector)	1D array of labels or values to predict. Shape (n_samples,).
Train-test split	Dividing data into a training set and a held-out test set.
Training set	The data the model learns from. Typically 70-80% of total data.
Test set	Data held out for honest evaluation. Model never sees it during training.
Generalisation	How well a model performs on unseen data. The real goal of ML.
Baseline	The simplest possible model. Real models must beat this to be useful.
scikit-learn (sklearn)	Python ML library. Consistent fit/predict interface for all algorithms.
model.fit(X, y)	Train the model on the data. Finds the best parameters.

<code>model.predict(X)</code>	Use the trained model to make predictions on new data.
<code>model.score(X, y)</code>	Evaluate the model. Returns accuracy for classifiers, R^2 for regressors.
Accuracy	Proportion of correct predictions. $(TP+TN)/Total$.
Precision	Of all positive predictions, what fraction were correct? $TP/(TP+FP)$
Recall	Of all actual positives, what fraction did we catch? $TP/(TP+FN)$
F1 Score	Harmonic mean of precision and recall. Good for imbalanced classes.
Confusion matrix	Table showing TP, TN, FP, FN counts for a classifier.
RMSE	Root Mean Squared Error. Average prediction error in same units as target.
R^2 (R-squared)	Proportion of target variance explained by the model. 1.0 = perfect.
<code>random_state=42</code>	Fixes random seed for reproducibility. Same split every run.
<code>stratify=y</code>	Keeps class proportions equal in train and test sets.
hyperparameter	A setting you choose before training. Not learned from data.

SECTION HW Homework — Submit Before Day 14

Create `day13_homework.py` and push to GitHub before midnight. This is the first Phase 2 homework. Every exercise uses the sklearn pattern: import, instantiate, fit, predict, evaluate.

Exercise 1 — Three Datasets, Same Workflow (Beginner)

Apply the complete 8-step workflow to three different sklearn built-in datasets: `load_iris`, `load_wine`, and `load_breast_cancer`. For each: load the data, check shape and class distribution, perform `train_test_split` (80/20, stratified), train a `DummyClassifier` (most_frequent strategy), print training and test accuracy, and write one comment explaining what the baseline accuracy means for that dataset.

Exercise 2 — Different Split Sizes (Beginner-Intermediate)

Using the Iris dataset, experiment with four different test sizes: 0.1, 0.2, 0.3, 0.4. For each split: train a `DummyClassifier`, record training accuracy and test accuracy. Print a summary table. Write a comment explaining: does the split size affect the dummy baseline? Why or why not? What would happen with a real model?

Exercise 3 — Confusion Matrix Deep Dive (Intermediate)

Create a simulated cancer screening scenario: 200 patients, 20 have cancer (class 1), 180 do not (class 0). Simulate a model that catches 18 of the 20 cancer cases but also has 15 false alarms. Build the confusion matrix manually using numpy arrays (do not use a classifier yet — manually

construct `y_true` and `y_pred`). Compute precision, recall, F1, and accuracy. Answer in comments: for this problem, which metric matters most and why?

Exercise 4 — Regression Baseline (Intermediate)

Generate a regression dataset using `make_regression` (`n_samples=300`, `n_features=3`, `noise=25`, `random_state=42`). Split 80/20. Train a `DummyRegressor` (`strategy=mean`). Compute RMSE, MAE, and R^2 on the test set. Then manually compute what 'always predict the mean' gives for five specific test samples. Verify your manual calculation matches sklearn's output.

Exercise 5 — Build and Evaluate on Real Data (Intermediate)

Load the cleaned Titanic dataset from your Day 8 homework. Select numerical features only (`Pclass`, `Age`, `SibSp`, `Parch`, `Fare`). Set `Survived` as the target `y`. Perform `train_test_split` (80/20, `stratified`, `random_state=42`). Train a `DummyClassifier` with `strategy='most_frequent'`. Print the baseline accuracy. Then compute and print the full classification report. Write 3 sentences interpreting the results.

Stretch Challenge — Tier A students

Implement your own `train_test_split` function from scratch using only NumPy. Your function signature should be: `my_split(X, y, test_size=0.2, random_state=42)`. It should: shuffle indices randomly (using the seed), split according to `test_size`, return `X_train`, `X_test`, `y_train`, `y_test` in that order. Verify that your function gives the same shapes and approximately the same class distribution as sklearn's `train_test_split` on the Iris dataset. Then implement a `DummyClassifier` from scratch that predicts the most frequent class in `y_train` for every test sample.